

PART I

以下作業截圖整張都是有修改範圍

ucos_ii.h:

```
78  typedef struct task_para_set {
79      INT16U TaskID;
80      INT16U TaskArriveTime;
81      INT16U TaskExecutionTime;
82      INT16U TaskPeriodic;
83      INT16U TaskNumber;
84      INT16U TaskPriority;
85      INT16U lockR1;
86      INT16U lockR2;
87      INT16U unlockR1;
88      INT16U unlockR2;
89  }task_para_set;
```

```
431  OS_EVENT* R1;
432  OS_EVENT* R2;
433  INT16U R1_PRIO, R2_PRIO;
434  INT16U R_USED;
```

新增 lock 與 mutex 相關紀錄變數

os_tcb:

```
644      INT16U      TaskExecutionTime;
645      INT16U      TASKWorkLoad;
646      INT16U      TaskPeriodic;
647      INT16U      JobNum;
648      INT16U      Blocked;
```

紀錄被 block 時間

main():

```
141      R1 = OSMutexCreate(0, &err);
142      printf("ERR: %d\n", err);
143      R2 = OSMutexCreate(1, &err);
144      printf("ERR: %d\n", err);
```

創建 event 使用 0 1 當比所有 task 高的 prio

```

198 void task(void* p_arg) {
199     task_para_set* task_data;
200     task_data = p_arg;
201     int count = 0;
202     OS_ERR err;
203     while (1) {
204         if(DEBUG)
205             printf("Tick: %d, start from task%d\n", OSTime, task_data->TaskID);
206         count = OSTCBCur->TaskExecutionTime - OSTCBCur->TASKWorkLoad;
207         if (count == task_data->lockR1 && task_data->unlockR1 != 0) {
208             printf("%2d\tTRY LockResource\ttask(%2d)(%2d) %d %d\tR1\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, count, R_USED);
209
210             OSMutexPend(R1, 0, &err);
211             R_USED++;
212             printf("%2d\tLockResource\ttask(%2d)(%2d) %d %d\tR1\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, count, R_USED);
213             fprintf(Output_fp, "%2d\tLockResource\ttask(%2d)(%2d) R1\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum);
214         }
215
216         if (count == task_data->lockR2 && task_data->unlockR2 != 0) {
217             printf("%2d\tTRY LockResource\ttask(%2d)(%2d) %d %d\tR2\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, count, R_USED);
218
219             OSMutexPend(R2, 0, &err);
220             R_USED++;
221             printf("%2d\tLockResource\ttask(%2d)(%2d) %d %d\tR2\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, count, R_USED);
222             fprintf(Output_fp, "%2d\tLockResource\ttask(%2d)(%2d) R2\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum);
223         }
224
225         if (count == task_data->unlockR1 && task_data->unlockR1 != 0) {
226             printf("%2d\tTRY UnlockResource\ttask(%2d)(%2d)\tR1\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum);
227             R_USED--;
228             printf("%2d\tUnlockResource\ttask(%2d)(%2d) %d %d\tR1\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, err);
229             fprintf(Output_fp, "%2d\tUnlockResource\ttask(%2d)(%2d) R1\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum);
230             err = OSMutexPost(R1);
231         }
232
233         if (count == task_data->unlockR2 && task_data->unlockR2 != 0) {
234             printf("%2d\tTRY UnlockResource\ttask(%2d)(%2d)\tR2\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum);
235             R_USED--;
236             printf("%2d\tUnlockResource\ttask(%2d)(%2d) %d %d\tR2\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, err);
237             fprintf(Output_fp, "%2d\tUnlockResource\ttask(%2d)(%2d) R2\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum);
238             err = OSMutexPost(R2);
239         }
240
241         if(DEBUG)
242             printf("Tick: %d, end from task%d\n", OSTime, task_data->TaskID);
243         count++;
244         OSTimeDly(task_data->TaskPeriodic);
245         //OSTimeDly(0);
246     }
247 }
248
249

```

task 內部寫成到了指定執行段落會 lock and unlock

```

static void OS_SchedNew (void)
{
    #if OS_LOWEST_PRIO <= 63u
        if (R_USED != 0u) {
            return;
        }
    #endif
}

```

要排程時如果發現有人使用資源就取消返回

OSMutexPost():

```
701     pevent->OSEventCnt |= OS_MUTEX_AVAILABLE;          /* No, Mutex is now available
702     pevent->OSEventPtr  = (void *)0;
703     OS_EXIT_CRITICAL();
704     OS_Sched();
705     OS_TRACE_MUTEX_POST_EXIT(OS_ERR_NONE);
706     return (OS_ERR_NONE);
```

每次 OSMutexPost()時本來只有有人在等待資源才排程，這裡我讓它最後一定會排程來讓同秒內解鎖跟排程兩者都會發生

OSTimeTick():

```
1007     ptcb = OSTCBList;                                /* Point at first TCB in TCB list
1008     while (ptcb->OSTCBPrio != OS_TASK_IDLE_PRIO) {    /* Go through all TCBs in TCB list
1009         OS_ENTER_CRITICAL();
1010         if (ptcb->OSTCBPrio < OSPrioCur && ptcb->TASKWorkLoad>0)
1011             ptcb->Blocked++;
```

如果當下 TASK 比自己低優先，block 時間+1

OSIntExit():

```
732     printf("%2d\tCompletion\ttask(%2d)\ttask(%2d)\t%td\t%td\t%td\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBCur->OSTCBID, OSTCBCur->JobNum + 1
733             , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly, OSTCBCur->Blocked, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly - OSTCBCur->TaskExecutionTime - OSTCBCur->Blocked);
734     fprintf(Output_fp, "%2d\tCompletion\ttask(%2d)\ttask(%2d)\t%td\t%td\t%td\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBCur->OSTCBID, OSTCBCur->JobNum + 1
735             , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly, OSTCBCur->Blocked, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly - OSTCBCur->TaskExecutionTime - OSTCBCur->Blocked);
736     OSTCBCur->JobNum += 1;
737     OSTCBCur->Blocked = 0;
```

OSTaskSwHook()

```
571     if (OSTCBCur->OSTCBPrio == 63) {
572         printf("%2d\tPreemption\ttask(%2d)\ttask(%2d)\t%td\n", OSTime, OSTCBCur->OSTCBPrio, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum);
573         fprintf(Output_fp, "%2d\tPreemption\ttask(%2d)\ttask(%2d)\t%td\n", OSTime, OSTCBCur->OSTCBPrio, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum);
574     }
575     else if (OSTCBHighRdy->OSTCBPrio == 63) {
576         printf("%2d\tCompletion\ttask(%2d)\ttask(%2d)\t%td\t%td\t%td\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBPrio
577             , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly, OSTCBCur->Blocked, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly - OSTCBCur->TaskExecutionTime - OSTCBCur->Blocked);
578         fprintf(Output_fp, "%2d\tCompletion\ttask(%2d)\ttask(%2d)\t%td\t%td\t%td\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBPrio
579             , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly, OSTCBCur->Blocked, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly - OSTCBCur->TaskExecutionTime - OSTCBCur->Blocked);
580         OSTCBCur->JobNum += 1;
581         OSTCBCur->Blocked = 0;
582     }
583     else {
584         if (OSTCBCur->TASKWorkLoad > 0 && OSTCBCur->OSTCBIDly != 0) {
585             printf("%2d\tPreemption\ttask(%2d)\ttask(%2d)\t%td\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum);
586             fprintf(Output_fp, "%2d\tPreemption\ttask(%2d)\ttask(%2d)\t%td\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum);
587         }
588         else {
589             printf("%2d\tCompletion\ttask(%2d)\ttask(%2d)\t%td\t%td\t%td\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum
590                 , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly, OSTCBCur->Blocked, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly - OSTCBCur->TaskExecutionTime - OSTCBCur->Blocked);
591             fprintf(Output_fp, "%2d\tCompletion\ttask(%2d)\ttask(%2d)\t%td\t%td\t%td\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum
592                 , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly, OSTCBCur->Blocked, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBIDly - OSTCBCur->TaskExecutionTime - OSTCBCur->Blocked);
593             OSTCBCur->JobNum += 1;
594             OSTCBCur->Blocked = 0;
595         }
596     }
```

更改輸出資訊為題目要求

PART II

ucos_ii.h

```
645     INT16U      TaskExecutionTime;
646     INT16U      TASKWorkLoad;
647     INT16U      TaskPeriodic;
648     INT16U      JobNum;
649     INT16U      Blocked;
650     INT16U      Ceil;
```

OSTCB 結構中新增 Ceil 紀錄目前值

OSTaskCreateExt():

```
397     task_para_set* para = (task_para_set*)p_arg;
398     OSTCBPrioTbl[prio]->OSTCBDly += para->TaskArriveTime;
399     OSTCBPrioTbl[prio]->TaskExecutionTime = para->TaskExecutionTime;
400     OSTCBPrioTbl[prio]->TaskPeriodic = para->TaskPeriodic;
401     OSTCBPrioTbl[prio]->JobNum = 0;
402     OSTCBPrioTbl[prio]->Blocked = 0;
403     OSTCBPrioTbl[prio]->Ceil = prio;
```

初始為本身優先度

InputFile():

```
176     for (int m = 0; m < TASK_NUMBER; m++) {
177         TaskParameter[m].TaskPriority = 3+m*3;
178     }
179
180     for (int i = 0; i < TASK_NUMBER; i++) {
181         printf("Task[%d] ID: %d, ArriveTime: %d, ExecutionTime: %d, Periodic: %d, Priority: %d\n", i, TaskParameter[i].TaskID, TaskParameter[i].TaskArriveTime, TaskParameter[i].TaskExecutionTime, TaskParameter[i].TaskPeriodic, TaskParameter[i].TaskPriority);
182         if (TaskParameter[i].unlockR1 != 0 && TaskParameter[i].TaskPriority < R1_Prio) {
183             R1_Prio = TaskParameter[i].TaskPriority-1;
184         }
185         if (TaskParameter[i].unlockR2 != 0 && TaskParameter[i].TaskPriority < R2_Prio) {
186             R2_Prio = TaskParameter[i].TaskPriority-2;
187         }
188     }
189     printf("R1_Prio: %d, R2_Prio: %d\n", R1_Prio, R2_Prio);
190     fclose(fp);
```

將優先度設為題目要求，並找出兩資源的 ceiling

main():

```
141     R1 = OSMutexCreate(R1_Prio, &err);
142     printf("ERR: %d\n", err);
143     R2 = OSMutexCreate(R2_Prio, &err);
144     printf("ERR: %d\n", err);
```

創建資源 EVENT

Task():

```
283 while (1) {
284     if(DEBUG)
285         printf("Tick: %d, start from task%d\n", OSTime, task_data->TaskID);
286     old_prio = OSTCBCur->OSTCBPrio;
287     count = OSTCBCur->TaskExecutionTime - OSTCBCur->TASKWorkLoad;
288     if (count == task_data->lockR1 && task_data->unlockR1 != 0) {
289         printf("X2d\tTRY LockResource\ttask(%2d)(%2d) %d %d\tR1\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, count, R_USED);
290         OSMutexPend(R1, 0, &err);
291         if (R_USED > R1_Prio)
292             R_USED = R1_Prio;
293         printf("X2d\tlockResource\ttask(%2d)(%2d)\tR1\t%d to %d\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, OSTCBCur->Ceil, R1_Prio);
294         fprintf(Output_fp, "X2d\tlockResource\ttask(%2d)(%2d)\tR1\t%d to %d\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, OSTCBCur->Ceil, R1_Prio);
295         OSTCBCur->Ceil = R1_Prio;
296     }
297     if (count == task_data->lockR2 && task_data->unlockR2 != 0) {
298         printf("X2d\tTRY LockResource\ttask(%2d)(%2d) %d %d\tR2\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, count, R_USED);
299         OSMutexPend(R2, 0, &err);
300         if (R_USED > R2_Prio)
301             R_USED = R2_Prio;
302         printf("X2d\tlockResource\ttask(%2d)(%2d)\tR2\t%d to %d\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, OSTCBCur->Ceil, R2_Prio);
303         fprintf(Output_fp, "X2d\tlockResource\ttask(%2d)(%2d)\tR2\t%d to %d\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, OSTCBCur->Ceil, R2_Prio);
304         OSTCBCur->Ceil = R2_Prio;
305     }
306     if (count == task_data->unlockR1 && task_data->unlockR1 != 0) {
307         printf("X2d\tTRY UnlockResource\ttask(%2d)(%2d)\tR1\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum);
308         if ((OS_TCB*)R2->OSEventPtr != (void*)0)
309             R_USED = R2_Prio;
310         else
311             R_USED = 63;
312         if ((OS_TCB*)R2->OSEventPtr == OSTCBCur)
313             OSTCBCur->Ceil = R2_Prio;
314         else
315             OSTCBCur->Ceil = OSTCBCur->OSTCBPrio;
316         printf("X2d\tUnlockResource\ttask(%2d)(%2d)\tR1\t%d to %d\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, R1_Prio, OSTCBCur->Ceil);
317         fprintf(Output_fp, "X2d\tUnlockResource\ttask(%2d)(%2d)\tR1\t%d to %d\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, R1_Prio, OSTCBCur->Ceil);
318         err = OSMutexPost(R1);
319     }
320     if (count == task_data->unlockR2 && task_data->unlockR2 != 0) {
321         printf("X2d\tTRY UnlockResource\ttask(%2d)(%2d)\tR2\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum);
322         if ((OS_TCB*)R1->OSEventPtr != (void*)0)
323             R_USED = R1_Prio;
324         else
325             R_USED = 63;
326         if ((OS_TCB*)R1->OSEventPtr == OSTCBCur)
327             OSTCBCur->Ceil = R1_Prio;
328         else
329             OSTCBCur->Ceil = OSTCBCur->OSTCBPrio;
330         printf("X2d\tUnlockResource\ttask(%2d)(%2d)\tR2\t%d to %d\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, R2_Prio, OSTCBCur->Ceil);
331         fprintf(Output_fp, "X2d\tUnlockResource\ttask(%2d)(%2d)\tR2\t%d to %d\n", OSTime, OSTCBCur->OSTCBId, OSTCBCur->JobNum, R2_Prio, OSTCBCur->Ceil);
332         err = OSMutexPost(R2);
333     }
334 }
```

實作 LOCK UNLOCK 並使用 R_USED 紀錄目前的最高 Ceil，在 LOCK 時往上提高，UNLOCK 時降回去

OS_SchedNew():

```
static void OS_SchedNew (void)
{
    #if OS_LOWEST_Prio <= 63u /* See if we support up to 64 tasks */
    {
        INT8U y, temp;
        y = OSUnMapTbl[OSRdyGrp];
        temp = (INT8U)((y << 3u) + OSUnMapTbl[OSRdyTbl[y]]);
        if ((temp != 63 && (temp < R_USED || ((OS_TCB*)R1->OSEventPtr == OSTCBPrioTbl[temp] || (OS_TCB*)R2->OSEventPtr == OSTCBPrioTbl[temp]))) || temp == 63)
            OSPrioHighRdy = (INT8U)((y << 3u) + OSUnMapTbl[OSRdyTbl[y]]);
    }
    #endif
}
```

要排程時檢查想換上去的 TASK 有沒有高於最高 Ceil，或者它本身是持有資源的或它是 IDLE TASK 才允許。

其他部分在 PART I 已實作完成

PART III

使用 **NPCS** 時，當任務進入 **CRITICAL SECTION** 使用共享資源時，不允許被其他任務搶佔，即使有更高優先權的任務 **ready**，直到執行到離開臨界區（釋放資源）為止。因此一旦擁有資源便能安全執行到需要資源的工作全處理完，因為任務在使用資源時無法被切換，不會同時多任務互相等待，由此避免 **DEADLOCK**。

優點:實作很簡單

缺點: 系統反應速度降低：任務可能卡住 **CPU**，降低即時性，並導致整體系統排程效率不佳。

使用 **CPP** 時，若取用某資源則會把自己的優先度提高到使用這資源的最高優先度再高一點，代表所有其他想用這資源的 **TASK** 都無法插它隊，假設有更高優先的 **TASK** 出現，也不會干擾到其執行因為沒有共用資源問題，由於其他資源不會跟它有互相等待資源的問題，因此避免了 **DEADLOCK**。

優點: 有助於減少群體阻塞效應，並提升提升效率與公平性

缺點:實作較複雜，並且系統負擔較高